

Guia completa del proyecto AML + Credit Risk Reporting

Documento de estudio para entrevista

AML Interview Demo

Contents

1	Guia completa del proyecto AML + Credit Risk Reporting	1
1.1	Resumen en una frase	2
1.2	Para que sirve	2
1.3	Que problema de negocio simula	2
1.4	Alcance real del proyecto	2
1.5	Estructura general del repositorio	3
1.6	Parte 1: Proyecto principal AML	3
1.7	Datos de entrada AML	4
1.8	Flujo de ejecucion AML	4
1.9	Script: src/ingest_data.py	4
1.10	Script: src/clean_data.py	5
1.11	Script: src/generate_features.py	5
1.12	Script: src/detect_alerts.py	5
1.13	Reglas AML implementadas	6
1.14	Reason codes	7
1.15	reporting.py	7
1.16	Parte 2: Capa de Credit Risk Reporting	8
1.17	Datos de entrada de credito	8
1.18	Que representa cada columna	9
1.19	Script: src/credit_risk_metrics.py	10
1.20	Calculos de credito	10
1.21	Funciones de credito en reporting.py	11
1.22	Parte 3: Dashboard Streamlit	12
1.23	Parte 4: Demo web Next.js	13
1.24	SQL	13
1.25	Carpeta experimental	13
1.26	Tests	14
1.27	Como ejecutar todo desde cero	14
1.28	Outputs importantes	14
1.29	Decisiones tecnicas importantes	15
1.30	Limitaciones	16
1.31	Como defender el proyecto en entrevista	16
1.32	Preguntas tipicas de entrevista y respuestas cortas	16

1 Guia completa del proyecto AML + Credit Risk Reporting

Esta guia explica el proyecto de forma sencilla y completa para poder estudiarlo y defenderlo en una entrevista.

El proyecto tiene dos ideas principales:

1. Un proyecto principal de **AML Transaction Monitoring Analytics** hecho en Python, Pandas y Streamlit.

2. Una capa pequeña y educativa de **Credit Risk Reporting** anadida encima, sin sustituir el proyecto AML.

No es un sistema bancario real. Es un proyecto académico/personal con datos sintéticos para demostrar que sabes limpiar datos, generar métricas, aplicar reglas explicables, crear alertas, medir calidad de datos y presentar resultados en dashboards.

1.1 Resumen en una frase

El proyecto transforma datos sintéticos de cuentas, transacciones y préstamos en alertas AML, KPIs de reporting, controles de calidad de datos y una pequeña capa de riesgo de crédito para entrevista.

1.2 Para que sirve

Sirve para practicar y demostrar conocimientos de:

- Python aplicado a datos.
- Pandas.
- Limpieza y normalización de CSV.
- Feature engineering sencillo.
- Reglas de detección explicables.
- Alertas AML.
- Reason codes.
- KPIs de reporting.
- Data quality.
- Dashboard BI con Streamlit.
- Demo web de entrevista con Next.js.
- Conceptos básicos de riesgo de crédito: NPL, PD, EAD, LGD, Expected Loss, recovery rate y stress testing simple.

1.3 Que problema de negocio simula

En un banco o entidad financiera hay dos necesidades muy comunes:

1. Monitorizar transacciones para detectar comportamientos inusuales relacionados con blanqueo de capitales.
2. Monitorizar préstamos para entender morosidad, exposición, pérdida esperada y señales tempranas de deterioro.

Este proyecto no decide si alguien ha cometido fraude ni si un cliente va a incumplir. Solo crea una capa analítica sencilla para priorizar revisión humana.

1.4 Alcance real del proyecto

El alcance principal es educativo:

- Datos ficticios.
- Reglas fijas.
- Cálculos transparentes.
- Outputs en CSV.
- Dashboard para explicar resultados.
- Demo web para entrevista.

No incluye:

- Modelos regulatorios reales.
- Decisiones automáticas.
- Bases de datos productivas.
- Airflow/dbt productivo en el flujo principal.
- Scoring ML nuevo para la capa de crédito.

- SHAP nuevo para credito.
- Altman Z-score.
- VaR complejo.
- Modelos macroeconomicos reales.

1.5 Estructura general del repositorio

AML/

```
|-- app.py
|-- README.md
|-- requirements.txt
|-- data/
|   |-- raw/
|   |   |-- accounts.csv
|   |   |-- transactions.csv
|   |   |-- loans.csv
|   |-- processed/
|   |   |-- accounts_ingested.csv
|   |   |-- transactions_ingested.csv
|   |   |-- accounts_clean.csv
|   |   |-- transactions_clean.csv
|   |   |-- account_features.csv
|   |   |-- transaction_features.csv
|   |   |-- credit_risk_metrics.csv
|-- outputs/
|   |-- alerts_sample.csv
|-- src/
|   |-- ingest_data.py
|   |-- clean_data.py
|   |-- generate_features.py
|   |-- detect_alerts.py
|   |-- credit_risk_metrics.py
|   |-- reporting.py
|-- sql/
|   |-- 01_create_tables.sql
|   |-- 02_account_features.sql
|   |-- 03_detection_rules.sql
|-- aml-interview-demo/
|   |-- src/
|   |   |-- components/demo-app.tsx
|   |   |-- lib/aml-engine.ts
|   |   |-- data/fixtures/
|-- docs/
|-- experimental/
```

1.6 Parte 1: Proyecto principal AML

AML significa **Anti-Money Laundering**, es decir, prevencion de blanqueo de capitales.

La parte AML del proyecto intenta responder esta pregunta:

Dadas unas cuentas y transacciones sinteticas, que patrones podrian ser raros y deberian revisarse?

El proyecto no dice "esto es delito". Dice "esto merece revision".

1.7 Datos de entrada AML

Los datos principales estan en:

- `data/raw/accounts.csv`
- `data/raw/transactions.csv`

1.7.1 `accounts.csv`

Representa cuentas o clientes ficticios.

Sirve para tener contexto de las cuentas que participan en las transacciones.

1.7.2 `transactions.csv`

Representa movimientos entre cuentas.

Normalmente contiene informacion como:

- identificador de transaccion,
- fecha/hora,
- cuenta emisora,
- cuenta receptora,
- importe,
- moneda,
- tipo de transaccion,
- pais o informacion geografica segun el caso.

El pipeline usa estas transacciones para generar features y alertas.

1.8 Flujo de ejecucion AML

El flujo principal se ejecuta con estos comandos:

```
python src/ingest_data.py
python src/clean_data.py
python src/generate_features.py
python src/detect_alerts.py
python src/credit_risk_metrics.py
streamlit run app.py
```

El orden importa:

1. Primero se ingieren datos.
2. Luego se limpian.
3. Despues se generan features.
4. Luego se detectan alertas.
5. Despues se calculan metricas de credito.
6. Finalmente se abre el dashboard.

1.9 Script: `src/ingest_data.py`

Este script carga los CSV crudos desde `data/raw/`.

Su papel es dejar una primera copia procesada en `data/processed/`.

Outputs principales:

- `data/processed/accounts_ingested.csv`
- `data/processed/transactions_ingested.csv`

Como explicarlo:

La ingesta separa el dato crudo del dato que empieza a entrar en el pipeline. Asi mantengo trazabilidad entre origen y procesamiento.

1.10 Script: `src/clean_data.py`

Este script limpia los datos ya ingeridos.

Hace trabajo tipico de calidad de datos:

- Normaliza columnas.
- Convierte fechas.
- Convierte importes a formato numerico.
- Elimina duplicados o registros invalidos.
- Trata registros con datos faltantes o inconsistentes.

Outputs principales:

- `data/processed/accounts_clean.csv`
- `data/processed/transactions_clean.csv`

Como explicarlo:

Antes de calcular alertas necesito que fechas, importes, IDs y categorias sean fiables. Si no limpio antes, las metricas y reglas pueden salir mal.

1.11 Script: `src/generate_features.py`

Este script crea variables analiticas a partir de las transacciones.

Una feature es una variable derivada. Por ejemplo:

- numero de transacciones por cuenta,
- importe total enviado o recibido,
- actividad en ventanas temporales,
- numero de contrapartes,
- patrones de comportamiento.

Outputs principales:

- `data/processed/account_features.csv`
- `data/processed/transaction_features.csv`

Como explicarlo:

No quiero analizar solo filas aisladas. Quiero convertir transacciones en comportamiento por cuenta, porque las alertas AML suelen depender de patrones.

1.12 Script: `src/detect_alerts.py`

Este script aplica reglas explicables para generar alertas.

Output principal:

- `outputs/alerts_sample.csv`

Cada alerta incluye:

- alert_id
- cuenta afectada,
- regla activada,
- severidad,
- razon,
- metricas de soporte,
- ventana temporal de deteccion.

Como explicarlo:

Las alertas no son conclusiones finales. Son casos priorizados para que un analista revise evidencia.

1.13 Reglas AML implementadas

1.13.1 Smurfing / structuring

Busca varias transferencias salientes pequenas en una ventana corta.

Idea de negocio:

En vez de hacer una transferencia grande, alguien podria dividir el importe en varias operaciones menores para evitar controles.

Por que es explicable:

- Se puede ver la cuenta.
- Se puede ver la ventana temporal.
- Se pueden ver los importes.
- Se puede ver cuantas operaciones activaron la regla.

1.13.2 Fan-in

Busca una cuenta que recibe fondos desde muchos originadores distintos en poco tiempo.

Idea de negocio:

Una cuenta podria estar funcionando como cuenta recolectora.

1.13.3 Fan-out

Busca una cuenta que envia fondos a muchos beneficiarios distintos en poco tiempo.

Idea de negocio:

Una cuenta podria estar dispersando dinero rapidamente hacia varias cuentas destino.

1.13.4 High-risk geography

Busca operaciones relacionadas con paises o geografias marcadas como de mayor riesgo en la demo.

Idea de negocio:

Algunas jurisdicciones requieren revision reforzada en contextos AML.

Importante:

En este proyecto la lista es sintetica y educativa. No es una lista regulatoria real.

1.14 Reason codes

Los reason codes son explicaciones cortas y consistentes para cada alerta.

Están definidos en `src/reporting.py`.

Ejemplos:

- SMURFING -> Potential structuring pattern.
- FAN_IN -> Repeated transactions in short period.
- HIGH_RISK_GEOGRAPHY -> High-risk rule match.

Para entrevista:

Un reason code ayuda a que el analista entienda rápidamente por qué existe una alerta. Es importante porque evita dashboards opacos.

1.15 reporting.py

`src/reporting.py` agrupa funciones reutilizables para el dashboard.

Funciones AML importantes:

- `load_demo_tables()`
- `add_alert_business_fields()`
- `data_quality_summary()`
- `calculate_reporting_metrics()`
- `build_alert_review_table()`

1.15.1 load_demo_tables()

Carga tablas procesadas:

- cuentas limpias,
- transacciones con features,
- alertas.

1.15.2 add_alert_business_fields()

Añade campos útiles para negocio:

- `rule_triggered`
- `reason_code`
- `status`

1.15.3 data_quality_summary()

Revisa problemas en `transactions.csv`, por ejemplo:

- campos clave vacíos,
- IDs duplicados,
- fechas inválidas,
- importes negativos o cero,
- categorías no esperadas.

1.15.4 calculate_reporting_metrics()

Calcula KPIs AML:

- total de transacciones,
- total de alertas,
- tasa de alertas,
- importe bajo alerta,
- media de importe alertado,
- cuentas unicas,
- issues de calidad de datos,
- alertas por severidad,
- alertas por regla,
- alertas en el tiempo,
- top cuentas por importe alertado.

1.15.5 build_alert_review_table()

Construye una tabla para revisar evidencia de alertas.

Relaciona:

- alerta,
- cuenta,
- ventana temporal,
- transacciones que apoyan la alerta.

Esto es importante porque en una entrevista puedes decir:

No solo muestro un KPI. Tambien puedo bajar al detalle de evidencia transaccional.

1.16 Parte 2: Capa de Credit Risk Reporting

La capa de credito se anadio encima del proyecto principal.

No sustituye AML.

No crea un proyecto nuevo.

No usa modelos complejos.

Su objetivo es educativo:

Poder explicar conceptos basicos de riesgo de credito en entrevista usando datos y formulas simples.

1.17 Datos de entrada de credito

Archivo:

- data/raw/loans.csv

Columnas:

- loan_id
- customer_id
- product_type
- origination_date
- outstanding_balance

- days_past_due
- pd
- ead
- lgd
- recovered_amount
- status

1.18 Que representa cada columna

1.18.1 loan_id

Identificador unico del prestamo.

1.18.2 customer_id

Identificador del cliente asociado al prestamo.

1.18.3 product_type

Tipo de producto:

- Personal Loan,
- Credit Card,
- Auto Loan,
- Mortgage,
- Working Capital,
- Equipment Loan.

1.18.4 origination_date

Fecha de origen del prestamo.

1.18.5 outstanding_balance

Saldo pendiente.

1.18.6 days_past_due

Dias de retraso en el pago.

Es una columna clave para morosidad.

1.18.7 pd

Probability of Default.

Es la probabilidad de default o incumplimiento.

En este proyecto es un valor sintetico, no calculado por un modelo real.

1.18.8 ead

Exposure at Default.

Es la exposicion estimada si ocurre default.

En la demo ya viene como columna para mantenerlo simple.

1.18.9 lgd

Loss Given Default.

Porcentaje de la exposicion que se perderia si ocurre default.

1.18.10 recovered_amount

Importe recuperado.

Sirve para calcular recovery rate.

1.18.11 status

Estado simple del prestamo:

- CURRENT,
- DELINQUENT,
- NPL.

1.19 Script: src/credit_risk_metrics.py

Este script lee:

- data/raw/loans.csv

Y genera:

- data/processed/credit_risk_metrics.csv

1.20 Calculos de credito

1.20.1 delinquency_bucket

Clasifica prestamos por dias de retraso:

- CURRENT: 0 dias de retraso.
- 1-30 DPD: entre 1 y 30 dias.
- 31-60 DPD: entre 31 y 60 dias.
- 61-90 DPD: entre 61 y 90 dias.
- 90+ DPD: mas de 90 dias.

DPD significa **Days Past Due**.

1.20.2 npl_flag

Marca si un prestamo es NPL.

Formula:

`npl_flag = days_past_due > 90`

NPL significa **Non-Performing Loan**.

En palabras simples:

Un prestamo NPL es un prestamo con deterioro serio por retraso superior a 90 dias.

1.20.3 early_warning_flag

Marca prestamos con retraso relevante, pero que aun no son NPL.

Formula:

```
early_warning_flag = days_past_due >= 30 and days_past_due <= 90
```

Sirve como indicador temprano.

Como explicarlo:

Antes de que un prestamo sea NPL, quiero detectar senales de deterioro para priorizar seguimiento.

1.20.4 expected_loss

Calcula perdida esperada.

Formula:

```
Expected Loss = PD * EAD * LGD
```

Ejemplo conceptual:

- Si PD es 10%,
- EAD es 10.000,
- LGD es 50%,

entonces:

```
Expected Loss = 0.10 * 10,000 * 0.50 = 500
```

1.20.5 recovery_rate

Calcula tasa de recuperacion observada.

Formula:

```
recovery_rate = recovered_amount / ead
```

Solo se calcula cuando ead > 0.

1.20.6 stressed_pd

Aplica un multiplicador de stress a la PD.

Formula:

```
stressed_pd = pd * stress_multiplier
```

Con limite maximo:

```
stressed_pd <= 1
```

Por defecto:

```
stress_multiplier = 1.5
```

1.20.7 stressed_expected_loss

Recalcula perdida esperada usando la PD estresada.

Formula:

```
stressed_expected_loss = stressed_pd * ead * lgd
```

1.21 Funciones de credito en reporting.py

Se anadieron funciones reutilizables:

- `load_credit_risk_table()`
- `credit_data_quality_summary()`
- `calculate_credit_risk_metrics()`

1.21.1 `load_credit_risk_table()`

Carga el CSV de prestamos y aplica los calculos de credito.

Tambien permite cambiar el `stress_multiplier`.

1.21.2 `credit_data_quality_summary()`

Hace controles de calidad sobre `loans.csv`.

Detecta:

- columnas obligatorias faltantes,
- campos clave vacios,
- IDs duplicados,
- fechas de origen invalidas,
- exposiciones invalidas,
- PD o LGD fuera de rango,
- `recovered_amount` mayor que EAD.

Como defenderlo:

En riesgo, un problema de calidad de datos tambien es riesgo operacional, porque puede distorsionar KPIs y decisiones.

1.21.3 `calculate_credit_risk_metrics()`

Calcula KPIs de credito:

- `total_loans`
- `total_ead`
- `npl_count`
- `npl_rate`
- `early_warning_count`
- `expected_loss`
- `stressed_expected_loss`
- `average_pd`
- `average_lgd`
- `average_recovery_rate`
- `data_quality_issues_detected_credit`

1.22 Parte 3: Dashboard Streamlit

Archivo principal:

- `app.py`

El dashboard se llama:

Risk Reporting & BI Demo

Sirve para presentar los resultados de forma visual.

Incluye:

- filtros,
- KPIs,
- graficos,
- tablas,
- revision de alertas,
- calidad de datos,
- capa de riesgo de credito,
- guion de entrevista.

1.23 Parte 4: Demo web Next.js

Carpeta:

- aml-interview-demo/

Esta demo es independiente del dashboard Streamlit, pero vive dentro del mismo repositorio.

Esta hecha con:

- Next.js,
- React,
- TypeScript,
- Recharts,
- Cytoscape,
- Framer Motion,
- lucide-react.

Su objetivo es mas visual e interactivo para entrevista.

Importante:

La demo web no sustituye al pipeline Python. Simula visualmente conceptos y evidencia para explicar el proyecto.

1.24 SQL

Carpeta:

- sql/

Archivos:

- 01_create_tables.sql
- 02_account_features.sql
- 03_detection_rules.sql

Sirven como practica o referencia SQL.

No son necesarios para ejecutar el flujo principal Python simplificado.

Como explicarlo:

Incluyo SQL para demostrar que entiendo como se podria llevar esta logica a tablas y consultas, aunque el flujo principal de la demo corre con CSV y Pandas.

1.25 Carpeta experimental

Carpeta:

- experimental/

Contiene trabajos avanzados o archivados:

- graph analytics,
- ML scoring,
- orchestration,
- dbt,
- Airflow,
- SQL investigations,
- documentos antiguos.

Importante:

La carpeta experimental no forma parte del flujo principal simplificado. Esta separada para no sobrecargar el proyecto principal.

1.26 Tests

Archivo:

- tests/test_reporting.py

Comprueba principalmente:

- calculo correcto de alert rate,
- deteccion de issues de calidad de datos,
- generacion de reason codes.

Comando:

```
python -m unittest discover -s tests -p "test_*.py"
```

1.27 Como ejecutar todo desde cero

Desde la raiz del repo:

```
pip install -r requirements.txt
```

```
python src/ingest_data.py
python src/clean_data.py
python src/generate_features.py
python src/detect_alerts.py
python src/credit_risk_metrics.py
```

```
streamlit run app.py
```

Para la demo web:

```
cd aml-interview-demo
npm install
npm run dev -- -p 3107
```

1.28 Outputs importantes

1.28.1 data/processed/accounts_ingested.csv

Cuentas despues de la ingesta.

1.28.2 data/processed/transactions_ingested.csv

Transacciones despues de la ingesta.

1.28.3 data/processed/accounts_clean.csv

Cuentas limpias.

1.28.4 data/processed/transactions_clean.csv

Transacciones limpias.

1.28.5 data/processed/account_features.csv

Features agregadas por cuenta.

1.28.6 data/processed/transaction_features.csv

Transacciones enriquecidas con variables utiles.

1.28.7 outputs/alerts_sample.csv

Alertas AML generadas por reglas.

1.28.8 data/processed/credit_risk_metrics.csv

Prestamos enriquecidos con buckets, NPL, early warning, Expected Loss, recovery rate y stress.

1.29 Decisiones tecnicas importantes**1.29.1 Por que CSV**

Porque el objetivo es que el proyecto sea facil de ejecutar y defender.

No necesitas levantar bases de datos ni servicios externos.

1.29.2 Por que Pandas

Porque es una herramienta comun para analisis de datos y permite mostrar claramente transformaciones.

1.29.3 Por que reglas explicables

Porque para una entrevista junior es mejor poder explicar cada alerta que meter un modelo complejo opaco.

1.29.4 Por que Streamlit

Porque permite convertir analisis Python en dashboard rapidamente.

1.29.5 Por que Next.js para la demo web

Porque permite una experiencia visual mas pulida para entrevista.

1.29.6 Por que la capa de credito es simple

Porque el objetivo es explicar conceptos, no simular un motor regulatorio real.

1.30 Limitaciones

Limitaciones honestas que puedes mencionar:

- Los datos son sinteticos.
- Las reglas AML son simples.
- No hay revision humana real.
- No hay conexion a sistemas bancarios.
- No hay modelo regulatorio real.
- La capa de credito usa PD, EAD y LGD ya dados, no estimados por un modelo.
- El stress test es solo multiplicar PD, no un escenario macroeconomico real.
- La demo web simula parte de la experiencia para entrevista.

1.31 Como defender el proyecto en entrevista

Puedes decir:

Este proyecto simula una capa analitica de riesgo. Primero ingiero y limpio datos sinteticos de cuentas y transacciones. Despues genero features para pasar de datos de operaciones a comportamiento por cuenta. Luego aplico reglas AML explicables como smurfing, fan-in, fan-out y geografias de riesgo. Cada alerta tiene reason code y evidencia para revision. Encima anadi una capa pequena de riesgo de credito con prestamos sinteticos, donde calculo buckets de morosidad, NPL >90 dias, early warnings, PD, EAD, LGD, Expected Loss, recovery rate y un stress test simple. Todo se presenta en Streamlit y tambien hay una demo web para explicar el flujo en entrevista. Es un proyecto educativo, no un sistema bancario real.

1.32 Preguntas tipicas de entrevista y respuestas cortas

1.32.1 Por que usaste reglas y no ML?

Porque el objetivo era explicabilidad. En AML, una alerta debe poder justificarse con evidencia y reason codes.

1.32.2 Que es una alerta?

Una senal de que una cuenta o transaccion merece revision. No es una conclusion final.

1.32.3 Que es data quality en este proyecto?

Validar que los datos tienen columnas obligatorias, fechas validas, importes positivos, IDs no duplicados y categorias esperadas.

1.32.4 Que es NPL?

Un prestamo non-performing. En esta demo se marca como NPL si tiene mas de 90 dias de retraso.

1.32.5 Que es Expected Loss?

La perdida esperada de credito:

$PD * EAD * LGD$

1.32.6 Que aporta el stress test?

Permite ver que pasa si la probabilidad de default empeora. Es simple y educativo.

1.32.7 Que parte es mas importante?

La trazabilidad: desde CSV crudo hasta KPI, alerta, reason code, evidencia y visualizacion.